ESAI Premium Dashboard - Full PRD and Implementation

Date: 2026-05-07

Source prototype: esai-premium-redesign-2.html

Audience: Codex / implementation agent, product owner, frontend engineer, backend engineer

Status: UI/UX prototype implemented; production backend integration pending

1. Product Summary

ESAI Premium Dashboard is an academic competition workflow platform for Indonesian students, university competition teams, essay/KTI writers, and academic users. The product helps users manage competitions, guidebooks, files, AI agents, writing outputs, citations, calendar deadlines, and analytics in one structured workflow.

The prototype is a high-fidelity HTML/React design artifact. It is not yet a production app. The production implementation should use a modern Next.js App Router app with Supabase persistence, storage, auth, and AI model execution via CLI and API-key-based providers.

2. Product Goals

- Reduce confusing workflows by separating each major job into a clear section.
- Make competition progress visible through dashboard cards and a dependency-aware stage pipeline.
- Make agent work file-based, so each agent consumes specific input files and produces specific output files.
- Let users choose whether agent inputs come from previous agent outputs or user-uploaded files.
- Let users edit outputs in a document-style workspace, not only through chat.
- Let users validate citations by comparing generated output against uploaded journal PDFs.
- Keep AI assistant contextual, available, and non-overlapping with the active work surface.
- Provide a Devs area for style profile, custom agents, settings, and BYOK model configuration.
- Persist all real user, competition, file, chat, and agent state through Supabase.

3. Primary Users

- Indonesian university students joining essay, KTI, PKM, PIMNAS, Gemastik, or similar competitions.
- Academic writing teams that need structured ideation, research, drafting, design, and review.
- Users who need AI assistance but must keep track of sources, uploaded PDFs, and generated files.
- Power users who want to customize agents, skills, required input files, and produced output files.

4. Current Prototype Scope

Implemented in the prototype:

- Dashboard with competition cards.
- Add Competition wizard.

- Calendar with month/week/day/list event manager.
- Competition Workbench with pipeline stages.
- Agent workspace with file dependency controls and A/B/C dummy agent questions.
- Stage output document preview, fullscreen editing, save, and version history.
- AI Assistant side section and chat composer.
- Devs menu with Style Builder, Agents, Settings, and BYOK Models.
- Style Builder document-mode flow with two source options:

- Upload PDF for analysis.

- Upload style_profile.md.

- Custom agent setup with skills .md, required input file, and produced output file.
- Validity-Checker with minimal side-by-side output/input comparison and Vault.
- Analytical Board and separate Analytical Dashboard.
- Theme controls: light/dark mode and accent color.

Not implemented yet:

- Real Supabase auth.
- Real Supabase database persistence.
- Real Supabase Storage uploads.
- Real AI execution.
- Real PDF parsing or OCR.
- Real journal citation verification.
- Real calendar persistence.
- Real CLI process execution from backend routes.

---

5. Information Architecture

5.1 Main Navigation

The production sidebar should include:

- Dashboard
- Calendar
- Devs
- Validity-Checker
- Final Outputs
- AI Assistant trigger
- User profile block that opens Analytical Board

Do not add a separate "My Competitions" page because Dashboard already owns that job.

5.2 Dashboard

Purpose: Select and manage competitions.

Required UI:

- Competition cards in portrait ratio.
- Poster/image area.
- Title, category, institution, progress, deadline, current stage.
- Add competition action.
- Future: edit/delete/archive via compact action menu.

Production behavior:

- Pull competition cards from Supabase.

- Card click sets selected competition and opens Workbench.
- Draft add-competition form data should persist locally while the modal is open and should be saved to Supabase only on submit.

5.3 Calendar

Purpose: Manage deadlines and stage events.

Current prototype behavior:

- Month, week, day, and list views.
- Search.
- Category and tag filters.
- New event modal.
- Event create/edit/delete dummy behavior.
- Upcoming events below the calendar.

Production behavior:

- Calendar events should be generated from:

- Competition deadline.
- Guidebook extracted timeline.
- User-created events.
- Agent-generated next-step tasks.

- Calendar must persist to Supabase.
- Calendar event rows should link back to competition and optionally stage.

5.4 Competition Workbench

Purpose: Run the staged competition workflow.

Core layout:

- Collapsible pipeline rail.
- Compact stage workspace.
- Agent chat.
- Stage output preview.
- Fullscreen output editor.
- AI Assistant as a side section when enabled.

Important UX rules:

- Pipeline dots must be connected.
- Collapsed rail must show all stages, including locked ones.
- Collapsed stage markers should show the first letter of each agent/stage.
- Selected stage should remain readable and show current progress with outline only.
- Avoid redundant top headers in the workspace because users can infer context from selected stage and file controls.

---

6. Stage Dependency Workflow

Agents are connected. Each agent demands specific input files before it can run and produces specific output files to unlock the next agent.

Stage Required Input Output Produced Notes

--- --- --- ---

Main Agent Onboarding 00_style_profile.md, guidebook, competition metadata 01_onboarding_map.md Can start from template buttons to reduce typing.

Ideation 01_onboarding_map.md 02_ideation_options.md Locked until onboarding output exists.

Research 02_ideation_options.md 03_research_brief.md Must follow selected idea.

Writing 03_research_brief.md, optional user essay draft 04_draft_essay.md Can use uploaded essay as context.

Flowchart 04_draft_essay.md 05_flowchart.png / .md spec Extracts concept flow after writing stabilizes.

Prototype 05_flowchart output 06_prototype_spec.html Produces prototype spec.

UI Design 06_prototype_spec.html 07_ui_mockup.html Produces design artifact.

Supervisor Draft + previous outputs 08_supervisor_review.md Final judge-style review.

6.1 Input File Provenance

Every stage input file can come from:

- Agent Output
- User Upload
- Devs
- System Template

The Input file button should show:

- File name.
- Provenance chip.
- Source detail, for example Agent Output - Ideation Agent or User Upload - Ari.

When clicked, it opens a picker with:

- From Vault tab.
- Uploaded by User tab.
- Upload .md button.
- Use as input action.

6.2 Agent Questions

Agents may ask user questions before continuing.

Prototype supports dummy A/B/C prompts:

- Agent question.
- Options A, B, C.
- Selected option highlight.
- Result summary.

Production behavior:

- The agent response may include a structured needs_user_choice object.
- UI opens a choice prompt only when the agent explicitly asks.
- The selected answer is saved to the agent run and used as context for continuation.

6.3 End-of-Stage Handoff

At the end of each stage, agent asks:

- Use this .md file for the next stage?
- Review first?
- Use this MD?

Production behavior:

- User approval should mark the output as approved.
- Only approved outputs should unlock the next stage by default.
- Users can override input using manual upload, but this should be visibly marked as user override.

---

7. Devs Menu

The Devs menu contains four sections:

- Style Builder
- Agents
- Settings
- BYOK Models

7.1 Style Builder

Purpose: Create or edit 00_style_profile.md.

Only two source options are allowed:

- Upload PDF for analysis.
- Upload style_profile.md.

The Style Builder should use document-mode layout like fullscreen stage output:

- Document surface is primary.
- AI is contextual support.
- Save button confirms the profile is saved to Vault.
- No paste sample option.

Production behavior:

- PDF upload goes to Supabase Storage.
- Backend extracts text from PDF.
- AI generates a style profile draft.
- User edits and saves.
- Saved profile becomes a file in Vault with role style_profile.

7.2 Agents

Purpose: Manage built-in and custom agents.

Custom agent requirements:

- Add Agent button.
- Agent name.
- Upload/select skills .md.
- Specific required input file.
- Produced output file.
- Optional description.
- Active/inactive state.

Production behavior:

- Custom agents are stored in Supabase.
- Their skill file becomes runtime prompt context.

- Input/output contract determines where the agent appears in workflow and what it can unlock.

7.3 Settings

Purpose: Configure workflow guardrails, output naming, default behavior.

Expected settings:

- Require approval before next stage unlock.
- Allow user-uploaded input override.
- File naming convention.
- Citation strictness.
- Default calendar creation behavior.

7.4 BYOK Models

Purpose: Let users bring their own AI model API key or use local CLI-backed model access.

Required controls:

- Provider selection.
- API key entry.
- Model list refresh.
- Default model selection.
- Reasoning effort defaults.
- Test connection button.

---

8. AI Assistant

8.1 Global Assistant Behavior

The AI Assistant is contextual. It should know where the user currently is:

- Dashboard
- Calendar
- Workbench stage
- Fullscreen output editor
- Validity-Checker comparison
- Style Builder document mode

The assistant must not cover the active work area as an overlay except for explicit mobile fallback. On desktop, it should appear as a side section or bottom section depending on context.

Current prototype behavior:

- On most pages, assistant appears as a side section.
- In Validity-Checker, assistant chat appears at the bottom when toggled, because the main goal is output-vs-input comparison.

8.2 Chat Composer Controls

Controls inside the chat box:

- Model dropdown.
- Reasoning effort dropdown:

- Low

- Medium

- High
- Extra High
  - Tools dropdown:
- Web search toggle.

8.3 Context Requirements

Every assistant message should include:

- Current route.
- Selected competition ID.
- Selected stage ID if inside Workbench.
- Selected input file and provenance.
- Selected output file.
- Selected citation text if in Validity-Checker.
- Current visible document if in fullscreen editor.

---

9. Stage Output and Document Editing

Stage output must feel like a document editor, not a chat log.

Required behavior:

- Open output fullscreen.
- Editable document surface.
- Save button.
- Save confirmation.
- Version history button.
- Version history opens separately, not always visible.
- AI edit assistance.

Data to persist:

- Current output content.
- Output versions.
- Author/source of each edit.
- Timestamp.
- Whether version is approved.

---

10. Validity-Checker

Purpose: Let users compare generated output/research output with uploaded journal PDFs and ask AI whether citations are actually supported.

The minimal product requirements are:

- User can upload documents.
- User can choose which output file and input PDF to compare.
- User can see a Vault that tells which files are user-uploaded and which are agent-generated.
- Output and Input Journal are compared side by side.
- AI chat can discuss the selected output/input PDFs and spot citation problems.

10.1 Vault

Vault should be minimal:

- One Vault button per relevant context.
- Popup, not inline expansion.
- Clicking outside closes popup.
- Light mode uses blur/translucency, not a black rectangle.
- Separate Output Vault and Input Vault:

- Output Vault shows agent output files only.
- Input Vault shows user input files only.

10.2 Comparison

Required layout:

- Output document pane.
- Input Journal document pane.
- Same size containers.
- File selector button in Output pane.
- File selector button in Input Journal pane.
- Side-by-side layout, not vertical stack on desktop.
- Bottom AI chat appears when toggled or when selected citation needs checking.

10.3 Citation Check

User flow:

- User selects claim/citation text in output file.
- User opens AI chat or clicks citation-check action.
- AI compares selected claim with selected journal PDF.
- AI returns:

- Supported: highlight paragraph used as citation.
- Risk citation: explain why claim needs review.

Production note:

- Real implementation must parse PDFs, extract text, chunk journal content, and run retrieval over selected journal PDF before AI judgment.

---

11. Analytical Board and Analytical Dashboard

User-name block opens Analytical Board. Profile should not be a main sidebar item.

Analytical Board contains:

- Entry button to Analytical Dashboard.
- Process notifications.
- Website look/accent controls.
- Dark mode toggle.

Analytical Dashboard is separate and should include:

- KPI Header Row:

- Wins
- Podium
- Finalist
- Performance Score

- Wins Over Time chart.
- Outcome Mix donut.

- Strongest Fields progress bars.
- Suggestions to Improve.
- Log Competition Results.

Dashboard must obey light/dark mode and accent system.

---

12. Calendar Event Manager

The Calendar section has been normalized from a 21st.dev style event manager into current ESAI design.

Required behavior:

- Month / Week / Day / List views.
- Previous / Today / Next navigation.
- Search events.
- Filter by category and tag.
- Active filter chips.
- New Event modal.
- Edit/delete event.
- Upcoming events below calendar.
- Event chips use current accent system.

Production data:

- Competition deadlines.
- Guidebook extracted dates.
- User events.
- Agent-generated events.
- Review reminders.
- Submission milestones.

---

13. Supabase Production Architecture

13.1 Supabase Products Needed

Use:

- Supabase Auth for user login.
- Supabase Postgres for app data.
- Supabase Storage for uploaded files and generated artifacts.
- Row Level Security for all exposed tables.
- Optional Edge Functions for file parsing and AI calls if not handled in Next.js API routes.
- Realtime only if collaborative editing or live agent output streaming is needed.

13.2 Environment Variables

Frontend-safe:

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
NEXT_PUBLIC_DISABLE_AUTH=false
```

Server-only:

```
SUPABASE_SERVICE_ROLE_KEY=
OPENAI_API_KEY=
OPENROUTER_API_KEY=
ANTHROPIC_API_KEY=
GOOGLE_GENERATIVE_AI_API_KEY=
OPENCLAW_FORCE_CLI=1
OPENCLAW_PROMPT_TIMEOUT_MS=180000
```

Security rules:

- Never expose SUPABASE_SERVICE_ROLE_KEY to the browser.
- Never put secret model API keys in NEXT_PUBLIC_*.
- Use server routes for all agent execution and key-backed model calls.

13.3 Recommended Tables

profiles

Stores public user profile and preferences.

Columns:

- id uuid primary key references auth.users(id)
- email text
- display_name text
- plan text
- accent text
- theme text
- created_at timestamptz
- updated_at timestamptz

competitions

Stores competition cards and metadata.

Columns:

- id uuid primary key
- user_id uuid references auth.users(id)
- title text
- category text
- institution text
- status text
- progress int
- deadline date
- registration_link text
- current_stage_id text
- poster_file_id uuid
- created_at timestamptz
- updated_at timestamptz

competition_files

Stores all uploaded and generated files.

Columns:

- id uuid primary key

- competition_id uuid references competitions(id)
- user_id uuid references auth.users(id)
- file_name text
- file_role text
- file_source text
- stage_id text
- agent_id uuid
- storage_bucket text
- storage_path text
- mime_type text
- size_bytes bigint
- content_text text
- metadata jsonb
- approved boolean default false
- created_at timestamptz

Recommended file_role values:

- guidebook
- poster
- twibbon
- user_photo
- style_profile
- stage_input
- stage_output
- research_output
- final_output
- journal_pdf
- citation_evidence
- registration_link

Recommended file_source values:

- user_upload
- agent_output
- devs
- system_template

agents

Stores built-in and custom agents.

Columns:

- id uuid primary key
- user_id uuid references auth.users(id)
- compartment text
- stage_id text
- name text
- description text
- skill_file_id uuid references competition_files(id)
- required_input_role text
- produced_output_role text
- is_custom boolean

- enabled boolean
- metadata jsonb
- created_at timestamptz
- updated_at timestamptz

agent_runs

Stores stage execution runs.

Columns:

- id uuid primary key
- competition_id uuid references competitions(id)
- agent_id uuid references agents(id)
- stage_id text
- user_id uuid references auth.users(id)
- status text
- model_provider text
- model_id text
- reasoning_effort text
- input_file_ids uuid[]
- output_file_id uuid
- needs_user_choice jsonb
- selected_choice jsonb
- error text
- created_at timestamptz
- updated_at timestamptz

agent_messages

Stores chat history for global, stage, Devs, and validity assistant.

Columns:

- id uuid primary key
- competition_id uuid
- agent_id uuid
- stage_id text
- thread_type text
- role text
- content text
- model_provider text
- model_id text
- reasoning_effort text
- context jsonb
- created_at timestamptz

Recommended thread_type values:

- global
- stage
- devs
- validity
- output_editor
- style_builder

output_versions

Stores document edit history.

Columns:

- id uuid primary key
- file_id uuid references competition_files(id)
- user_id uuid references auth.users(id)
- version_number int
- content_text text
- change_summary text
- created_at timestamptz

calendar_events

Stores calendar data.

Columns:

- id uuid primary key
- competition_id uuid references competitions(id)
- user_id uuid references auth.users(id)
- title text
- description text
- start_time timestamptz
- end_time timestamptz
- category text
- color text
- tags text[]
- source text
- created_at timestamptz
- updated_at timestamptz

validity_checks

Stores citation verification results.

Columns:

- id uuid primary key
- competition_id uuid references competitions(id)
- user_id uuid references auth.users(id)
- output_file_id uuid references competition_files(id)
- journal_file_id uuid references competition_files(id)
- selected_claim text
- verdict text
- evidence_text text
- risk_reason text
- model_provider text
- model_id text
- created_at timestamptz

13.4 Storage Buckets

Recommended buckets:

- competition-files
- agent-outputs
- journal-pdfs
- profile-assets

Storage paths:

```
competition-files/{user_id}/{competition_id}/uploads/{file_id}-{filename}
agent-outputs/{user_id}/{competition_id}/{stage_id}/{file_id}-{filename}
journal-pdfs/{user_id}/{competition_id}/{file_id}-{filename}
profile-assets/{user_id}/{filename}
```

13.5 RLS Requirements

Enable RLS on every table.

Base rule:

- Users can only access rows where user_id = auth.uid().

For team support later:

- Add competition_members.
- Policies should allow members to access competition files and runs.

Do not use user-editable metadata for authorization. If roles are needed, store them in server-managed app metadata or membership tables.

13.6 Supabase Dashboard Setup Checklist

In Supabase Dashboard:

- Create project.
- Enable Email auth provider.
- Create storage buckets.
- Run SQL migrations for tables.
- Enable RLS.
- Add table policies.
- Add storage policies.
- Copy project URL and publishable key into .env.local.
- Copy service role key into server-only .env.local.
- Test insert/select for competitions.
- Test upload/select/download for files.
- Test agent output save.
- Test calendar event save.
- Test Validity-Checker journal PDF upload.

---

14. API Route Architecture

Recommended Next.js App Router route groups:

```
src/app/api/competitions/route.ts
src/app/api/competitions/[id]/route.ts
src/app/api/files/route.ts
src/app/api/files/[id]/route.ts
src/app/api/agents/route.ts
src/app/api/agent-runs/route.ts
src/app/api/agent-chat/route.ts
src/app/api/calendar-events/route.ts
src/app/api/validity-checks/route.ts
src/app/api/models/route.ts
src/app/api/models/test/route.ts
```

14.1 Required Server Responsibilities

Server routes must:

- Authenticate user.
- Validate competition ownership.
- Use Supabase admin client only server-side.
- Validate file roles and source.
- Build agent context from selected files.
- Save outputs and versions.
- Call model provider or CLI.
- Return structured errors for UI.

---

15. AI Model Integration

The app should support two model access paths:

- CLI-backed execution.
- API-key-backed execution.

15.1 CLI Path

Use this when user wants local Codex/OpenClaw style execution.

Recommended model runner contract:

```
type ModelRunRequest = {
provider: "openclaw-cli"   "codex-cli";
model: string;
reasoningEffort: "low"   "medium"   "high"   "xhigh";
systemPrompt: string;
userMessage: string;
files: Array<{ name: string; content: string; role: string }>;
timeoutMs: number;
};
```

Recommended execution behavior:

- Build prompt server-side.
- Spawn CLI with child_process.execFile, not shell string concatenation.
- Pass prompt through stdin or safe argument array.
- Enforce timeout.
- Capture stdout/stderr.
- Parse JSON if CLI supports --json.
- Save raw run log to agent_runs.metadata.

- Save output file if successful.

Example CLI environment:

```
OPENCLAW_FORCE_CLI=1
OPENCLAW_PROMPT_TIMEOUT_MS=180000
```

Stable execution pattern from prior local work:

```
openclaw agent --json --session-id <id> --message <message> --timeout 600
```

Production route should not assume the shell alias exists. Resolve the executable path or use the absolute local entrypoint.

15.2 API Key Path

Use this when user brings a provider API key.

Supported provider design:

- OpenAI-compatible API.
- OpenRouter.
- Anthropic.
- Google Gemini.
- Other BYOK providers later.

The user enters API key in BYOK Models. The app should:

- Store encrypted key server-side or use a secure secret vault.
- Never expose key to browser after save.
- Test the key through /api/models/test.
- Fetch model list through /api/models.
- Let user choose model and reasoning effort.

Recommended byok_keys table:

- id uuid primary key
- user_id uuid references auth.users(id)
- provider text
- label text
- encrypted_api_key text
- default_model text
- created_at timestamptz
- updated_at timestamptz

Do not store plaintext API keys in the database unless there is a deliberate encryption layer.

15.3 Model Selection UX

Every chat composer should include:

- Model dropdown.
- Reasoning effort dropdown.
- Tools dropdown.
- Web search toggle.

Model selection must be required before agent generation. If no model is selected, disable send/generate and show a visible notice.

15.4 AI Context Builder

Every AI call should include:

- User profile and style profile if available.
- Competition metadata.
- Guidebook summary.
- Stage ID.
- Required input files.
- Output file being edited.
- User-selected choice answer if relevant.
- Chat history summary.
- Validity selected claim and selected journal PDF if relevant.

---

16. PDF and Citation Verification Implementation

For Validity-Checker production:

- User uploads output/research/final paper.
- User uploads journal PDFs from bibliography.
- Backend extracts text from PDFs.
- System chunks journal PDFs by page/paragraph.
- User selects claim text in output.
- AI retrieves relevant journal chunks.
- AI returns:

- Supported.

- Partially supported.

- Not supported / risk citation.

- UI highlights evidence paragraph when supported.
- Result is saved to validity_checks.

Recommended storage:

- Store original PDF in Storage.
- Store extracted text in competition_files.content_text or a separate file_chunks table.

Recommended optional file_chunks table:

- id uuid primary key
- file_id uuid references competition_files(id)
- page_number int
- chunk_index int
- content text
- embedding vector
- metadata jsonb

If semantic retrieval is needed, enable pgvector and store embeddings.

---

17. Frontend Implementation Plan for Codex

Recommended production stack:

- Next.js App Router.
- TypeScript.
- Tailwind CSS.
- shadcn/ui components.
- Supabase SSR client.
- Server routes for AI and file processing.

Recommended implementation order:

- Create Next.js app and theme tokens.
- Build AppShell and sidebar.
- Implement Supabase Auth and profile preferences.
- Implement Dashboard and Add Competition flow.
- Implement Supabase Storage file upload.
- Implement Workbench pipeline and stage output files.
- Implement document editor and output versions.
- Implement Devs Style Builder and Agents.
- Implement BYOK Models and model testing.
- Implement AI Assistant shell and chat persistence.
- Implement agent run route with CLI/API provider abstraction.
- Implement Calendar event manager.
- Implement Validity-Checker document comparison.
- Implement PDF parsing and citation verification.
- Implement Analytical Board/Dashboard.
- Run build, lint, route tests, and browser QA.

18. Suggested Production Folder Structure

```
src/
app/
api/
agent-chat/
agent-runs/
calendar-events/
competitions/
files/
models/
validity-checks/
dashboard/
layout.tsx
page.tsx
components/
app-shell/
calendar/
```

When coding in Codex:

- Create .env.local.
- Add Supabase URL and publishable key.
- Add service role key for server routes only.
- Create Supabase client helpers:
- browser client

- server client
- admin client
  - Add auth gate.
  - Add temporary dev bypass only if explicitly needed.
  - Create migrations.
  - Enable RLS.
  - Add storage policies.
  - Run npm run build.
  - Test:
- create competition
- upload guidebook
- save agent output
- open workbench
- save chat
- save calendar event
- upload journal PDF
- create validity check

20. Acceptance Criteria

P0:

- User can create/select a competition from Dashboard.
- User can upload guidebook/poster/files.
- User can run or simulate each agent stage in dependency order.
- Stage input can come from agent output or user-uploaded .md.
- Stage output can be edited, saved, versioned, and approved.
- Devs Style Builder can save 00_style_profile.md.
- Custom agents can define skills .md, required input, and produced output.
- AI Assistant can chat with current context and use selected model/effort.
- Validity-Checker can compare output and journal PDF side by side.
- Supabase persists competitions, files, outputs, messages, events, and validity checks.

P1:

- Calendar auto-generates events from guidebook timeline.
- Citation evidence is highlighted from parsed PDF content.
- Analytical Dashboard uses real competition performance data.
- BYOK keys are encrypted and model list is fetched dynamically.

P2:

- Team collaboration.
- Realtime agent streaming.
- Collaborative document editing.
- Full mobile optimization.

21. Risks and Mitigations

Risk Impact Mitigation

--- --- ---

Prototype is a large single HTML file Hard to maintain directly Rebuild as typed Next.js components.

AI workflows can become confusing Users lose trust Keep file provenance, approval prompts, and stage locks visible.

Citation verification may hallucinate Academic risk Require PDF retrieval evidence and show risk verdict when uncertain.

API keys can leak Security breach Store only server-side, encrypt BYOK keys, never expose service role.

Supabase schema drift Runtime bugs Use migrations and verification queries.

CLI execution hangs Bad UX Hard timeout and fallback provider path.

22. What Codex Should Do Next

If implementing this project from the prototype:

- Do not continue editing the giant HTML as production code.
- Create a Next.js + TypeScript implementation.
- Use the prototype only as visual/workflow reference.
- Start with Supabase schema and AppShell.
- Implement Dashboard -> Workbench -> Files -> AI routes before advanced analytics.
- Keep Validity-Checker UI minimal and document-first.
- Keep all AI calls server-side.
- Verify with npm run build after each major slice.

23. Reference Artifacts

- Main prototype: esai-premium-redesign-2.html
- Current Calendar-applied backup: esai-premium-redesign-2-calendar-event-manager-applied.html
- Prior PRD PDF: output/pdf/esai-premium-redesign-prd.pdf
- Prior PRD source: output/pdf/esai-premium-redesign-prd.html
- Recovered backup: esai-premium-redesign-2-db-recovered.html